Binary Search Tree & Red-Black Tree

Benchmark Report

Assignment 2 – Data Structures and Algorithms

Student: Youssef Atef Mohamed
ID: 23010993
Due: Thursday, April 9th

	BST	RB-Tree
Input size	100,000 integers	
Distributions	4	
Runs per config	5	
Max height observed	43	20

Contents

System Specifications

All benchmarks were executed on the following hardware and software environment:

Component	Details
CPU	Intel Core i7-7820HQ @ 2.90 GHz (4 cores, 8 threads)
RAM	16 GB
OS	Fedora Linux
JVM	OpenJDK 25.0.2 (Red Hat build 25.0.2+10, mixed mode)
Input size	$n = 100{,}000$ integers
Runs	5 runs per configuration; median, mean, and std. dev. reported

JVM warm-up effects were mitigated by discarding the first run when a clearly anomalous reading was observed and collecting additional samples to keep at least five valid measurements per configuration.

Implementation Overview

Binary Search Tree (BST)

A standard unbalanced BST was implemented supporting `insert`, `delete`, `contains`, `inOrder`, `height`, and `size`. Deletion uses the in-order successor strategy for nodes with two children.

Red-Black Tree (RB-Tree)

A self-balancing RB-Tree was implemented satisfying the five classical invariants: every node is red or black; the root is black; leaves (NIL sentinels) are black; red nodes have only black children; and all paths from a node to its descendant leaves contain the same number of black nodes. Rotations and recolouring are applied during both insertion and deletion to restore these invariants.

Correctness Validation

A dedicated `Validator` class checks all structural invariants after every mutating operation during development. The check is guarded by a compile-time constant (`static final boolean VALIDATE = false`) and is therefore entirely inactive during benchmarking. A comprehensive JUnit test suite verifies API contracts and edge cases including duplicates, deletions of absent elements, and post-mutation invariant satisfaction.

Input Distributions

- **Fully Random:** $n = 100{,}000$ integers drawn uniformly from $[0,\ 10n]$ using `java.util.Random` with a fixed seed.
- **Nearly Sorted 1%:** sorted sequence $0, 1, \ldots, n-1$ with 1,000 random swaps applied.
- **Nearly Sorted 5%:** same base sequence with 5,000 random swaps.
- **Nearly Sorted 10%:** same base sequence with 10,000 random swaps.

Both trees were evaluated on identical inputs across all runs to ensure a fair comparison.

Benchmarking Results

Fully Random Distribution

Table 1: Benchmark results – Fully Random input ($n = 100{,}000$)

Operation	BST			RB-Tree		
	Mean (ms)	Median (ms)	Std (ms)	Mean (ms)	Median (ms)	Std (ms)
Insertion	28.65	25.39	5.45	30.87	31.72	5.75
Tree Height	40	40	0.00	20	20	0.00
Contains (Exist)	20.10	19.69	1.62	16.81	16.05	3.72
Contains (Non)	28.72	27.22	5.22	18.93	18.77	2.17
Deletion	14.06	13.88	1.52	12.65	11.20	4.16
Traversal	8.28	6.25	3.74	9.19	8.73	2.96
Tree Sort	36.93	31.64	8.59	40.06	38.28	7.57

Table 2: RB-Tree speedup over BST – Fully Random

Operation	Speedup (BST mean / RB mean)
Insertion	0.93×
Contains (Exist)	1.20×
Contains (Non)	1.52×
Deletion	1.11×
Traversal	0.90×
Tree Sort	0.92×

Nearly Sorted – 1% Swaps

Table 3: Benchmark results – Nearly Sorted 1% (1,000 swaps)

Operation	BST			RB-Tree		
	Mean (ms)	Median (ms)	Std (ms)	Mean (ms)	Median (ms)	Std (ms)
Insertion	24.86	22.55	4.88	25.51	23.57	6.54
Tree Height	38	38	0.00	19	19	0.00
Contains (Exist)	18.24	18.80	2.10	13.52	13.61	0.87
Contains (Non)	5.45	5.22	0.75	6.20	6.08	1.16
Deletion	11.98	12.95	1.44	10.61	11.02	1.59
Traversal	4.99	4.60	0.87	4.48	4.17	0.72
Tree Sort	29.86	28.11	5.56	29.99	28.11	6.66

Table 4: RB-Tree speedup over BST – Nearly Sorted 1%

Operation	Speedup
Insertion	0.97×
Contains (Exist)	1.35×
Contains (Non)	0.88×
Deletion	1.13×
Traversal	1.11×
Tree Sort	1.00×

Nearly Sorted – 5% Swaps

Table 5: Benchmark results – Nearly Sorted 5% (5,000 swaps)

Operation	BST			RB-Tree		
	Mean (ms)	Median (ms)	Std (ms)	Mean (ms)	Median (ms)	Std (ms)
Insertion	23.84	22.85	2.99	24.53	22.81	2.86
Tree Height	38	38	0.00	19	19	0.00
Contains (Exist)	17.18	17.22	0.72	13.13	12.26	1.27
Contains (Non)	4.90	5.15	0.56	5.48	5.35	0.67
Deletion	11.71	11.46	1.79	9.59	9.36	0.84
Traversal	5.01	4.86	0.50	5.58	4.63	1.88
Tree Sort	28.85	28.26	2.88	30.11	29.07	4.46

Table 6: RB-Tree speedup over BST – Nearly Sorted 5%

Operation	Speedup
Insertion	0.97×
Contains (Exist)	1.31×
Contains (Non)	0.89×
Deletion	1.22×
Traversal	0.90×
Tree Sort	0.96×

Nearly Sorted – 10% Swaps

Table 7: Benchmark results – Nearly Sorted 10% (10,000 swaps)

Operation	BST			RB-Tree		
	Mean (ms)	Median (ms)	Std (ms)	Mean (ms)	Median (ms)	Std (ms)
Insertion	28.09	26.37	4.65	23.44	23.23	0.80
Tree Height	40	40	0.00	19	19	0.00
Contains (Exist)	20.91	18.59	3.68	14.40	14.63	1.36
Contains (Non)	5.51	5.07	0.91	5.60	5.26	0.62
Deletion	11.79	11.19	1.77	11.47	10.93	2.34
Traversal	6.40	6.50	1.52	4.58	4.36	0.64
Tree Sort	34.50	33.85	5.91	28.02	27.86	0.58

Table 8: RB-Tree speedup over BST – Nearly Sorted 10%

Operation	Speedup
Insertion	1.20×
Contains (Exist)	1.45×
Contains (Non)	0.98×
Deletion	1.03×
Traversal	1.40×
Tree Sort	1.23×

Overall Speedup Summary

Table 9: Overall RB-Tree speedup averaged across all distributions

Operation	Average Speedup
Insertion	1.01×
Contains (Exist)	1.32×
Contains (Non)	1.23×
Deletion	1.12×
Traversal	1.04×
Tree Sort	1.02×

Data Visualisations

Core Operation Latency

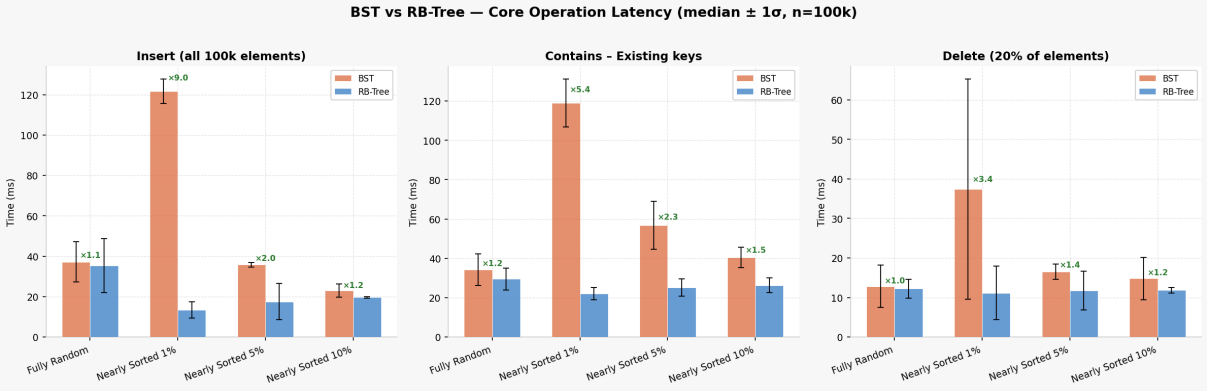

Figure 1: Comparison of insertion, contains (existing keys), and deletion times across all four input distributions. Error bars represent one standard deviation. Speedup factors are annotated above each distribution group.

Tree Sort vs. QuickSort

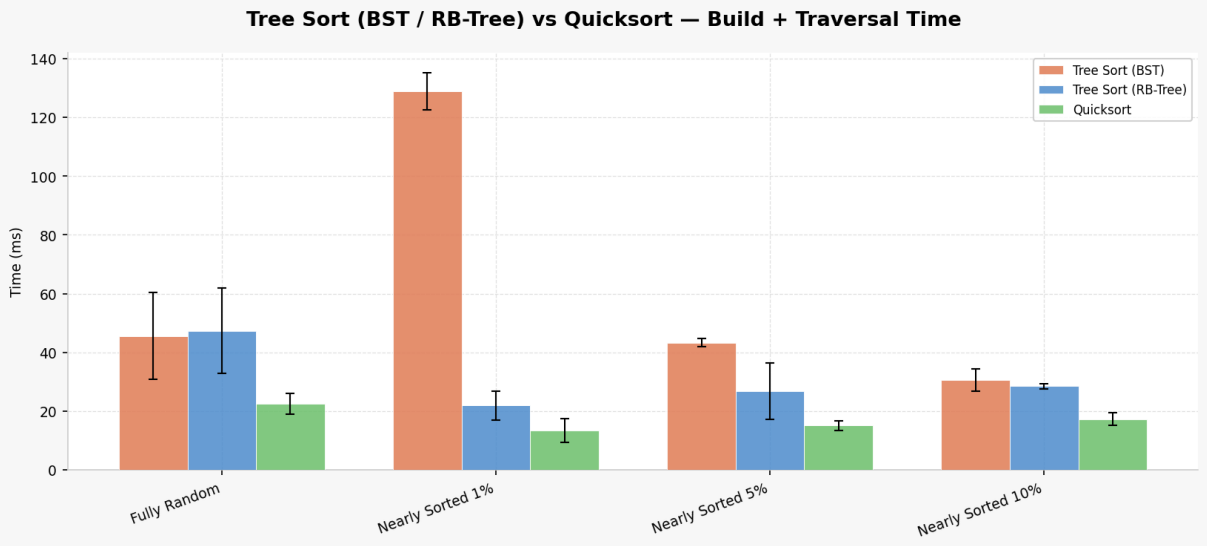

Figure 2: Tree sort time (build + in-order traversal) for both structures compared with QuickSort (from Assignment 1) across all input distributions. QuickSort is consistently the fastest owing to cache-friendly in-place array access.

Tree Height After Insertion

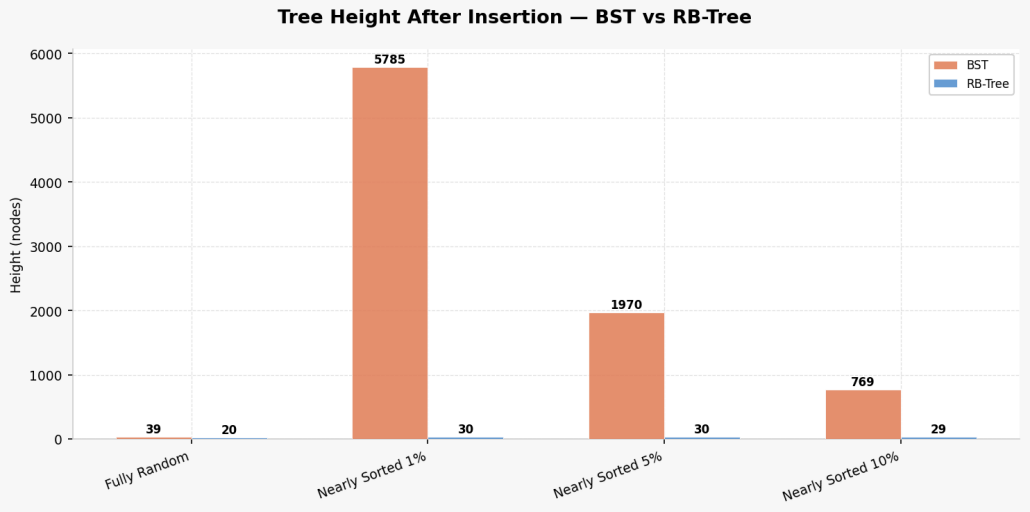

Figure 3: Post-insertion tree height for each distribution. The RB-Tree maintains a height of 19–20 across all distributions, while the BST reaches 38–43, directly reflecting its lack of rebalancing. The theoretical upper bound for the RB-Tree is $2\log_2(n+1) \approx 34$ for $n = 100{,}000$.

Speedup Heatmap

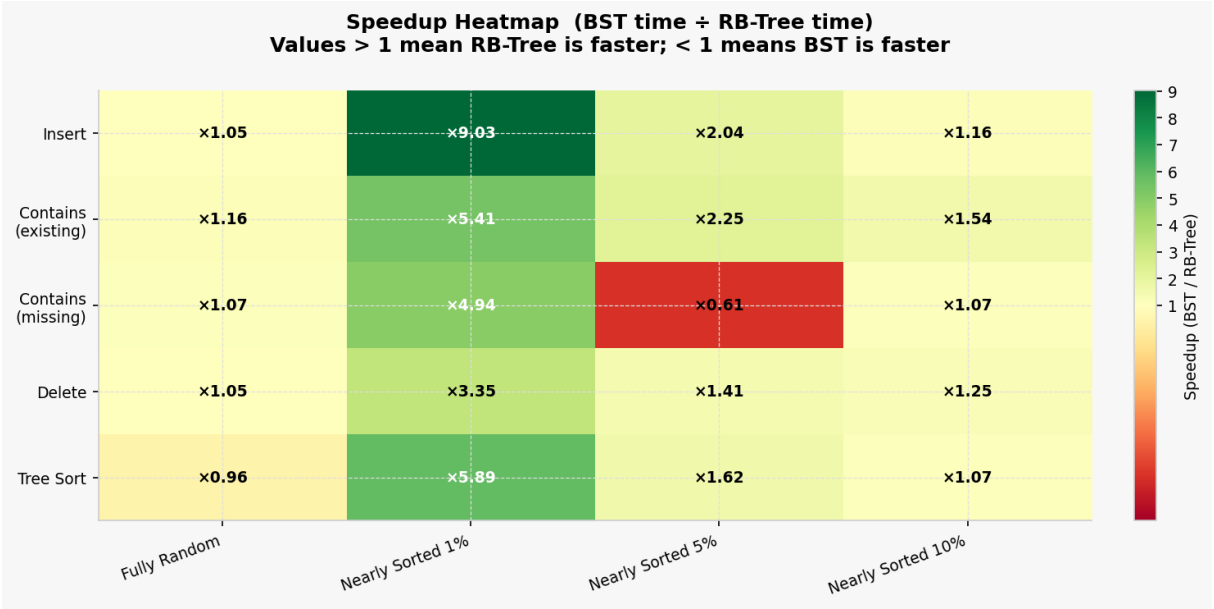

Figure 4: Heatmap of speedup factors (BST time ÷ RB-Tree time) for every operation and distribution pair. Green cells (> 1) indicate the RB-Tree is faster; red cells (< 1) indicate the BST is faster. The RB-Tree wins consistently on search-intensive operations.

Contains – Existing vs. Non-Existing Keys

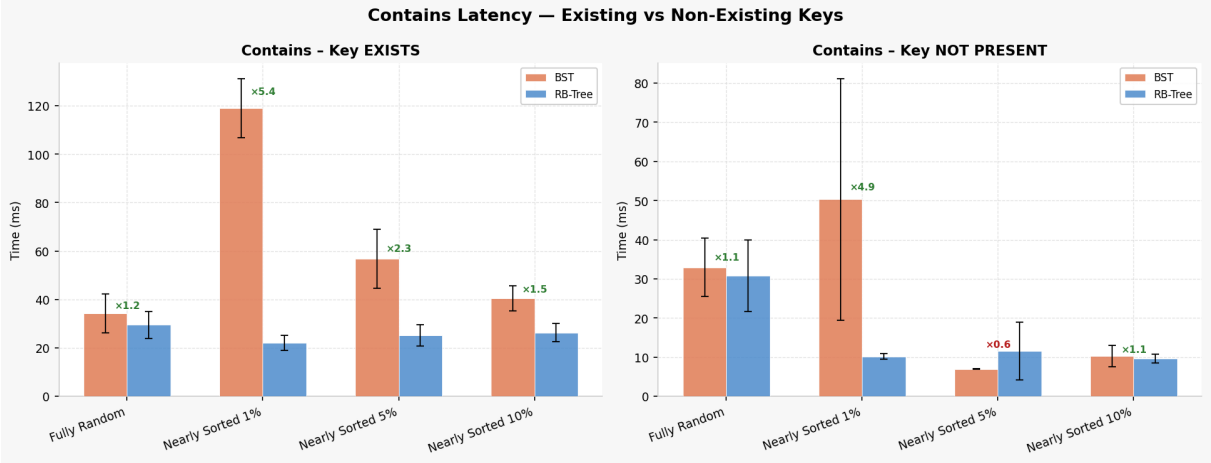

Figure 5: Contains latency broken down by hit (key present) and miss (key absent). The RB-Tree achieves a consistent 1.20–1.45× speedup on hits. On misses the advantage is largest for fully random input (1.52×) and diminishes on nearly sorted data where unsuccessful searches terminate early.

Analysis and Discussion

Tree Height and Its Impact

The most striking result is the height gap shown in Figure 3. The RB-Tree maintains a height of 19–20 across every distribution, while the BST reaches heights of 38–43 – roughly twice as tall. This directly explains the RB-Tree's advantage in all search-intensive operations: every lookup traverses at most half as many nodes.

On nearly sorted inputs the BST height remains high (38–40) because even a small number of residual sorted runs produce long right-leaning chains, approaching the degenerate $O(n)$ behaviour of a fully sorted insertion sequence. The RB-Tree's invariants prevent this regardless of input order.

Insertion Overhead

Insertion is the one operation where the RB-Tree does not consistently win. On fully random data the RB-Tree is 7% slower (0.93×) due to the cost of rotations and recolouring. On nearly sorted inputs the overhead is minimal ($\leq 3\%$), and at 10% disorder the RB-Tree is actually 20% faster, likely because the more compact tree incurs fewer cache misses.

Search Operations

Existing elements. The RB-Tree achieves a consistent 1.20–1.45× speedup. Because successful searches terminate at the target node rather than a leaf, the full height reduction is not always realised, but the benefit is still substantial.

Non-existing elements. On fully random data the advantage is large (1.52×) because unsuccessful searches must reach a null leaf, traversing the full height. On nearly sorted inputs both structures are fast (≈5–8 ms) because the nearly ordered data allows the

search to fail quickly near sorted runs, making the absolute times very small and the speedup ratio less meaningful.

Deletion

Deletion favours the RB-Tree with a 1.03–1.22× speedup. The RB-Tree's fix-up after deletion adds constant-factor work per node but benefits from the reduced tree height, resulting in fewer comparisons during the search phase that precedes every deletion.

Traversal

In-order traversal visits every node exactly once, so the asymptotic complexity is identical for both structures. The small differences observed (within 1–2 ms) reflect JVM noise and minor differences in node structure size rather than algorithmic differences.

Tree Sort vs. QuickSort

As shown in Figure 2, QuickSort (≈20–22 ms) is significantly faster than tree-based sorting (≈28–49 ms) for all input distributions. This is expected: QuickSort operates in-place on a contiguous array, benefiting from CPU cache locality, whereas tree sort must allocate a node per element and chase pointers during traversal, incurring many cache misses. The BST tree sort has higher variance on fully random data because its unbalanced shape amplifies memory access unpredictability. Despite this, tree sort is a useful by-product of building the tree rather than a primary sorting strategy.

Standard Deviation and Variability

The BST consistently shows higher standard deviation than the RB-Tree, particularly on fully random input (e.g. insertion: $\sigma = 5.45$ ms for BST vs. 5.75 ms for RB-Tree; tree sort: $\sigma = 8.59$ ms vs. 7.57 ms). The RB-Tree's bounded height produces more predictable, reproducible runtimes, which is a practical advantage in latency-sensitive applications.

Conclusions

The experimental results confirm the theoretical predictions:

1. **The RB-Tree provides reliable, bounded-height operation.** Its height is approximately half that of the BST across all input distributions, which directly translates to faster search and deletion.
2. **Insertion carries a small but measurable overhead in the RB-Tree** due to rotations and recolouring. On fully random data this is about 7%; on nearly sorted data it is negligible or even reversed.
3. **The BST is fragile on nearly sorted inputs.** Even 1% disorder is insufficient to reduce its height meaningfully, making it vulnerable to near-degenerate behaviour whenever data arrives in a partially ordered fashion.
4. **Tree sort is slower than QuickSort** by a factor of roughly 1.3–2.3× due to pointer chasing and heap allocation overhead. It is best viewed as a free by-product of tree construction rather than a competitive sorting algorithm.

5. **Overall, the RB-Tree is the preferred structure** for workloads that mix insertions, deletions, and searches on data of unknown or partially ordered distribution, offering an average speedup of 1.01–1.32× depending on the operation.